

Day 1: Database Fundamentals & Basic Data Manipulation

Welcome to your 30-Day SQL Mastery journey! Today we build the foundation: understanding what a database is, how tables are structured, and how to create, insert, and query data using the most essential SQL commands. By the end of this day, you'll be able to design a simple table, populate it, and retrieve meaningful information – skills you will use every day as a Data Analyst.

1. Learning Objectives

- Understand the role of a relational database management system (RDBMS) and how SQL Server fits in.
 - Differentiate between a database, a schema, and a table.
 - Identify and use common SQL Server data types.
 - Write **CREATE TABLE** statements to define new tables.
 - Insert rows using **INSERT INTO ... VALUES**.
 - Query data with **SELECT, WHERE, ORDER BY, DISTINCT**, and **TOP**.
 - Use column aliases to make output more readable.
 - Apply best practices for naming conventions and code formatting.
-

2. Concepts

Database Basics

Definition

A database is an organised collection of structured data. In the relational model, data is stored in tables (relations) that are linked by common columns. Microsoft SQL Server is a powerful RDBMS that uses Transact-SQL (T-SQL) to manage and query data.

Why it's used

Databases provide:

- Persistent storage
- Data integrity (via constraints)
- Concurrent access control
- Security
- Scalability

Business use cases

- E-commerce: customer orders, product inventory
- Healthcare: patient records, appointments
- Finance: transactions, loan applications
- HR: employee information, payroll

Syntax (high-level)

```
-- Create a database (optional; you often work in an existing one)
CREATE DATABASE CompanyDB;
GO

-- Use a specific database
USE CompanyDB;
GO
```

Best practices

- Use meaningful names for databases, tables, and columns.
- Store each database on appropriate filegroups for performance.
- Keep development and production environments separate.

Common mistakes

- Forgetting to switch to the correct database with **USE**.
- Using reserved words as object names without brackets.

Performance considerations

- Database design (normalisation) directly affects query speed.
- Choose appropriate file growth settings to avoid fragmentation.

Interview tips

- Be prepared to explain ACID properties and when to use a RDBMS vs NoSQL.
- Understand how SQL Server differs from other databases (e.g., **TOP** vs **LIMIT**).

Tables

Definition

A table is a collection of rows (records) and columns (attributes). Each column has a specific data type. Tables are the core storage unit in a relational database.

Why it's used

Tables organise data into a structured format that can be easily queried, updated, and related to other tables.

Business use cases

- **Customers** table: stores customer details.
- **Orders** table: stores order header information.
- **OrderItems** table: stores line items for each order.

Syntax

```
CREATE TABLE table_name (
    column1 datatype constraint,
    column2 datatype constraint,
```

```
...  
);
```

Best practices

- Choose a primary key for each table (e.g., an identity column).
- Use **NOT NULL** where values are mandatory.
- Keep column names concise but descriptive.

Common mistakes

- Not defining a primary key, leading to duplicate rows.
- Using overly large data types (e.g., **VARCHAR(MAX)** when not needed).

Performance considerations

- A well-defined primary key (clustered index) speeds up lookups.
- Avoid too many columns per table; normalise where appropriate.

Interview tips

- Know the difference between a table and a view.
- Understand when to use a temporary table vs a permanent one.

Data Types

Data Type	Description	Example Value
INT	Whole numbers (-2^{31} to $2^{31}-1$)	12345
BIGINT	Larger whole numbers	9876543210
DECIMAL(p, s)	Exact numeric with precision p and scale s	99.99
VARCHAR(n)	Variable-length string, max length n	'John Smith'
CHAR(n)	Fixed-length string, padded with spaces	'ABC'
DATE	Date only (no time)	'2026-07-08'
DATETIME	Date and time (accuracy ~3.33 ms)	'2026-07-08 14:30:00'
BIT	Boolean (0 or 1)	1 (true)
UNIQUEIDENTIFIER	Globally unique identifier (GUID)	'6F9619FF-...'

Why it's used

Data types enforce the kind of data that can be stored, ensuring correctness and efficient storage.

Business use cases

- Use **VARCHAR(50)** for a customer's first name.
- Use **DECIMAL(10, 2)** for monetary values to avoid rounding errors.

Best practices

- Choose the smallest data type that fits your data.
- For dates, prefer **DATE** or **DATETIME2** (more precision) over **DATETIME**.

Common mistakes

- Storing dates as strings – makes date arithmetic slow and error-prone.
- Using **VARCHAR(MAX)** unnecessarily; it uses special storage mechanisms.

Performance considerations

- Smaller data types mean more rows per page, reducing I/O.
- **VARCHAR** is generally better than **CHAR** for variable-length strings.

Interview tips

- Be aware of the difference between **VARCHAR** and **NVARCHAR** (Unicode).
- Explain when to use **DECIMAL** vs **FLOAT**.

CREATE TABLE

Definition

The **CREATE TABLE** statement defines a new table with its columns, data types, and optional constraints.

Why it's used

To set up the database schema before inserting any data.

Syntax

```
CREATE TABLE dbo.Employees (  
    EmployeeID INT IDENTITY(1,1) PRIMARY KEY,  
    FirstName VARCHAR(50) NOT NULL,  
    LastName VARCHAR(50) NOT NULL,  
    HireDate DATE NOT NULL,  
    Salary DECIMAL(10,2),  
    IsActive BIT DEFAULT 1  
);
```

Explanation

- **IDENTITY(1,1)** – auto-increment starting at 1, step 1.
- **PRIMARY KEY** – ensures uniqueness and creates a clustered index.
- **NOT NULL** – prohibits null values.
- **DEFAULT 1** – sets a default value if not provided.

Best practices

- Always specify a schema (e.g., **dbo**) for clarity.
- Add constraints like **CHECK**, **UNIQUE**, **FOREIGN KEY** for data integrity.

Common mistakes

- Forgetting to define a primary key.
- Not allowing enough length for text columns.

Performance considerations

- Primary keys should be narrow (e.g., `INT` or `BIGINT`).
- Consider index strategies when creating tables (we'll cover indexes later).

Interview tips

- Know how to alter a table after creation (`ALTER TABLE`).
 - Understand the difference between `IDENTITY` and sequences.
-

INSERT

Definition

`INSERT INTO` adds one or more rows to a table.

Why it's used

To populate tables with data – either initial data or new records.

Syntax

```
-- Insert a single row
INSERT INTO dbo.Employees (FirstName, LastName, HireDate, Salary)
VALUES ('John', 'Doe', '2026-07-01', 60000.00);

-- Insert multiple rows
INSERT INTO dbo.Employees (FirstName, LastName, HireDate, Salary)
VALUES ('Jane', 'Smith', '2026-07-02', 65000.00),
      ('Mike', 'Johnson', '2026-07-03', 62000.00);
```

Best practices

- Explicitly list the column names (avoids issues if column order changes).
- Use `GETDATE()` for the current date/time when appropriate.

Common mistakes

- Inserting values in the wrong order.
- Trying to insert a duplicate primary key value.

Performance considerations

- Use batch inserts for many rows to reduce round-trips.
- Consider `INSERT INTO ... SELECT` for copying large datasets.

Interview tips

- Know how to insert data from another table (**INSERT INTO ... SELECT**).
 - Understand identity insert (**SET IDENTITY_INSERT ON**).
-

SELECT, WHERE, ORDER BY, DISTINCT, TOP, Alias

SELECT – retrieves columns from a table.

WHERE – filters rows based on a condition.

ORDER BY – sorts the result set.

DISTINCT – returns unique rows.

TOP – limits the number of rows returned (SQL Server equivalent to **LIMIT**).

Alias – gives a temporary name to a column or table.

Syntax

```
SELECT TOP 5 DISTINCT
    FirstName AS [First Name],
    LastName AS [Last Name],
    Salary
FROM dbo.Employees
WHERE Salary > 50000
ORDER BY Salary DESC;
```

Explanation

- **TOP 5** – returns only the first 5 rows.
- **DISTINCT** – removes duplicate combinations (here, only distinct names).
- **AS [First Name]** – aliases the column for display.
- **WHERE** – filters to salaries > 50000.
- **ORDER BY Salary DESC** – sorts from highest to lowest salary.

Best practices

- Use **TOP** with **ORDER BY** to get consistent results.
- Use meaningful aliases for readability.

Common mistakes

- Forgetting **ORDER BY** when using **TOP** – results may be arbitrary.
- Using **DISTINCT** unnecessarily; it can slow queries.

Performance considerations

- **DISTINCT** may require a sort or hash operation – use only when needed.
- **TOP** with an index that supports the **ORDER BY** is efficient.

Interview tips

- Know the difference between **TOP** and **OFFSET-FETCH** (used for paging).
 - Explain how **DISTINCT** affects performance.
-

3. Examples

Simple Examples

1. Create a simple product table and insert a row

```
-- Create table
CREATE TABLE dbo.Products (
    ProductID INT IDENTITY(1,1) PRIMARY KEY,
    ProductName VARCHAR(100) NOT NULL,
    Price DECIMAL(10,2) NOT NULL,
    Stock INT NOT NULL
);

-- Insert a product
INSERT INTO dbo.Products (ProductName, Price, Stock)
VALUES ('Wireless Mouse', 25.99, 150);

-- Query all products
SELECT * FROM dbo.Products;
```

Line-by-line:

- **IDENTITY** generates unique ProductID automatically.
 - **INSERT** adds one row with specified values.
 - **SELECT *** returns all columns (use with caution in production).
-

Intermediate Examples

2. Insert multiple products and query with filters

```
-- Insert 3 products
INSERT INTO dbo.Products (ProductName, Price, Stock)
VALUES ('Keyboard', 45.50, 75),
      ('Monitor', 199.99, 30),
      ('USB Cable', 8.75, 200);

-- Get products with stock > 50, sorted by price descending, show top 2
SELECT TOP 2
    ProductName,
    Price,
    Stock AS [Current Stock]
FROM dbo.Products
```

```
WHERE Stock > 50
ORDER BY Price DESC;
```

Expected output:

ProductName	Price	Current Stock
Keyboard	45.50	75
Wireless Mouse	25.99	150

Explanation: The query filters for products with Stock > 50, sorts by Price high to low, and returns only the top 2.

Advanced Examples

3. Using DISTINCT with multiple columns and custom aliases

```
-- Suppose Products has a Category column (add later)
-- For now, demonstrate DISTINCT on Price and Stock
SELECT DISTINCT
    Price,
    Stock
FROM dbo.Products
ORDER BY Price;
```

Line-by-line:

- **DISTINCT** returns unique combinations of Price and Stock.
- Because there are no duplicates, all rows appear.
- **ORDER BY Price** sorts the distinct result.

Advanced scenario: You can use **TOP** with **PERCENT** to return a percentage of rows.

```
SELECT TOP 50 PERCENT
    ProductName,
    Price
FROM dbo.Products
ORDER BY Price DESC;
```

This returns half the rows (rounded up) with the highest prices.

4. Hands-on Practice

We'll use a **Sales** dataset for today's exercises. Run the following code to set up your practice environment:


```
CREATE DATABASE PracticeDB;
GO
USE PracticeDB;
GO

-- Create a Customers table
CREATE TABLE dbo.Customers (
    CustomerID INT IDENTITY(1,1) PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    City VARCHAR(50),
    JoinDate DATE DEFAULT GETDATE()
);

-- Create an Orders table
CREATE TABLE dbo.Orders (
    OrderID INT IDENTITY(1,1) PRIMARY KEY,
    CustomerID INT NOT NULL,
    OrderDate DATE NOT NULL DEFAULT GETDATE(),
    TotalAmount DECIMAL(10,2) NOT NULL
);

-- Insert sample customers
INSERT INTO dbo.Customers (FirstName, LastName, Email, City)
VALUES ('Alice', 'Brown', 'alice@example.com', 'New York'),
       ('Bob', 'Davis', 'bob@example.com', 'Los Angeles'),
       ('Charlie', 'Evans', 'charlie@example.com', 'Chicago'),
       ('Diana', 'Foster', 'diana@example.com', 'New York'),
       ('Eve', 'Garcia', 'eve@example.com', 'Los Angeles');

-- Insert sample orders
INSERT INTO dbo.Orders (CustomerID, OrderDate, TotalAmount)
VALUES (1, '2026-07-01', 150.00),
       (2, '2026-07-02', 89.99),
       (3, '2026-07-03', 200.50),
       (1, '2026-07-04', 45.20),
       (2, '2026-07-05', 320.00),
       (4, '2026-07-06', 75.00);
```

Now try these exercises:

Exercise 1 (Easy):

Write a query to retrieve all columns from the `Customers` table.

Exercise 2 (Easy):

Retrieve the first name, last name, and city of customers from New York.

Exercise 3 (Easy):

Get a list of unique cities from the `Customers` table.

Exercise 4 (Intermediate):

Retrieve the top 3 orders with the highest TotalAmount, showing OrderID, CustomerID, and TotalAmount, sorted descending.

Exercise 5 (Intermediate):

Find all orders placed after July 2, 2026, and show the OrderDate and TotalAmount, sorted by OrderDate ascending.

Exercise 6 (Intermediate):

Display the customer's first name and last name combined into a single column called "Full Name", along with their Email, for customers in Los Angeles.

Exercise 7 (Advanced):

Write a query that returns the top 2 most recent orders (by OrderDate) for each customer? (We'll cover joins later, but for now try using a subquery or a window function if you know – but not required yet; we'll revisit.)

Expected Outputs (for reference)

Exercise 1: All 5 rows from Customers.

Exercise 2: Alice Brown New York; Diana Foster New York.

Exercise 3: New York, Los Angeles, Chicago.

Exercise 4:

OrderID	CustomerID	TotalAmount
5	2	320.00
3	3	200.50
1	1	150.00

Exercise 5: Orders with OrderDate > '2026-07-02' (order 3,1,2,5? Actually > July 2 means July 3,4,5,6). Should return OrderID 3,1? Wait check: Orders inserted:

- 1: Jul 1
- 2: Jul 2
- 3: Jul 3
- 4: Jul 4
- 5: Jul 5
- 6: Jul 6

So > Jul 2 returns orders 3,4,5,6. Sorted ascending: Jul 3 (order 3), Jul 4 (order 4), Jul 5 (order 5), Jul 6 (order 6).

Exercise 6: Full Name and Email for City = 'Los Angeles':

Bob Davis bob@example.com; Eve Garcia eve@example.com.

5. Mini Challenge

Challenge:

You are given a table **Sales** with columns **SaleID**, **ProductName**, **SaleDate**, **Amount**, and **Quantity**. Write a single SQL query that does all of the following:

- Returns the top 3 distinct product names (by product name, not by count) that have at least one sale with a quantity greater than 10.
- Show the product names in alphabetical order.
- Exclude sales with amount less than 100.
- Display the product name and the maximum amount for that product (across all sales of that product).
- Rename the maximum amount column as **MaxAmount**.

*Hint: You may need to use **SELECT DISTINCT**, **TOP**, **WHERE**, **ORDER BY**, and an aggregate function – but you can solve it with a subquery or a **GROUP BY**; we will cover **GROUP BY** tomorrow. For today, try to think about it.*

Do not reveal the answer yet – try to solve it on your own.

6. Real Business Scenario

Scenario: E-commerce Inventory Management

Company: An online electronics retailer (similar to Best Buy).

Problem: The inventory team needs to quickly identify the top 5 most expensive products that are currently in stock, but only those that have been sold at least once in the last 30 days. They want a list of product names, prices, and current stock levels, sorted by price descending.

How SQL solves it:

We have two tables: **Products** (ProductID, ProductName, Price, Stock) and **OrderItems** (OrderItemID, OrderID, ProductID, Quantity, SaleDate).

The query will:

1. Join **Products** and **OrderItems** on ProductID.
2. Filter for sales within the last 30 days (using **DATEADD**).
3. Use **DISTINCT** to avoid duplicate products.
4. Filter for Stock > 0.
5. Sort by Price DESC.
6. Use **TOP 5** to get only the top 5.

We'll implement this query once we learn joins (Day 15). For now, understand that basic **SELECT** with filters and ordering is the foundation for such tasks.

7. Interview Questions

Basic

Q1: What is the difference between **DELETE** and **TRUNCATE**?

Answer: **DELETE** removes rows one by one, can have a **WHERE** clause, and triggers triggers. **TRUNCATE** removes all rows in a table, resets identity seed, and is faster because it deallocates data pages. Both are DML vs DDL? Actually **TRUNCATE** is DDL, **DELETE** is DML.

Q2: How do you select the top 5 rows from a table in SQL Server?

Answer: `SELECT TOP 5 * FROM table_name;`

Intermediate

Q3: What is the effect of `DISTINCT` on performance?

Answer: `DISTINCT` forces SQL Server to sort or hash the result set to remove duplicates, which can be resource-intensive on large datasets. Use it only when you genuinely need unique rows.

Q4: How do you avoid duplicate rows without using `DISTINCT`?

Answer: Use `GROUP BY` on all selected columns, or use a subquery with `ROW_NUMBER()` and filter on row number = 1.

Advanced

Q5: Explain the logical order of operations in a `SELECT` query.

Answer: The logical order is: `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY` → `OFFSET/FETCH` (or `TOP`). Understanding this helps write efficient queries and debug unexpected results.

Q6: Can you use `TOP` with a variable?

Answer: Yes, `SELECT TOP (@variable) * FROM table;` – but you must use parentheses.

8. Cheat Sheet

Common Commands:

Command	Description
<code>CREATE DATABASE db_name;</code>	Create a new database.
<code>USE db_name;</code>	Switch to the specified database.
<code>CREATE TABLE table (...);</code>	Define a new table.
<code>INSERT INTO table (cols) VALUES (...);</code>	Add rows.
<code>SELECT columns FROM table;</code>	Retrieve data.
<code>WHERE condition</code>	Filter rows.
<code>ORDER BY col ASC/DESC</code>	Sort results.
<code>DISTINCT</code>	Remove duplicates.
<code>TOP n</code>	Return first n rows.
<code>AS alias</code>	Rename column/table temporarily.

Data Type Quick Reference:

Type	Use for
<code>INT</code>	Whole numbers (up to 2 billion).

Type	Use for
DECIMAL(p, s)	Precise numbers like money.
VARCHAR(n)	Text up to n characters.
DATE	Date only.
DATETIME	Date and time.

Best Practices:

- Always list columns in **INSERT**.
- Use **NOT NULL** for mandatory columns.
- Provide a primary key.
- Use **TOP** with **ORDER BY** for predictable results.

9. Homework

Complete the following questions using the **PracticeDB** tables created earlier. Write the SQL query for each.

1. Retrieve all columns from the Orders table.
2. Retrieve only the OrderID and OrderDate for orders with TotalAmount greater than 100.
3. Get the first name, last name, and email of customers whose city is 'New York' or 'Chicago'.
4. Retrieve the list of all unique customer IDs from the Orders table.
5. Show the top 2 orders with the smallest TotalAmount (sorted ascending).
6. Display all orders sorted by OrderDate descending, and then by TotalAmount ascending within the same date.
7. Write a query that returns the OrderID, TotalAmount, and a column called "AmountCategory" that shows 'High' if TotalAmount > 200, 'Medium' if between 100 and 200, and 'Low' otherwise (use **CASE** – we'll cover tomorrow, but try to learn).
8. Retrieve the customers who have an email address (i.e., Email is not NULL).
9. Get the total number of customers (count) from the Customers table.
10. Insert a new customer into the Customers table with your own name and city.
11. Update the city of the customer with CustomerID = 3 to 'San Francisco'.
12. Delete the customer with CustomerID = 5 (if exists) – be careful.
13. Write a query to show the top 3 cities with the most customers (use **GROUP BY** with **COUNT** and **TOP** – we'll cover tomorrow, but try to think).
14. Retrieve the OrderID and a calculated column "DiscountedAmount" which is 90% of TotalAmount (i.e., 10% discount).
15. Write a query that returns the first name, last name, and total order amount for each customer? (We need join, later).
16. Find all orders that were placed in the month of July 2026 (use **YEAR** and **MONTH** functions).
17. Show the distinct combinations of City and CustomerID? (Not meaningful but practice).
18. Write a query that returns the OrderID, OrderDate, and TotalAmount, with the TotalAmount formatted to two decimal places (use **FORMAT** or **CAST**).
19. Retrieve the customers whose first name starts with 'A' or ends with 'e' (use **LIKE**).
20. Write a query that returns all columns from the Orders table but limits the result to 3 rows, skipping the first 2 rows? (Use **OFFSET-FETCH – ORDER BY ... OFFSET 2 ROWS FETCH NEXT 3 ROWS ONLY**).